

# Constraint Folding

複数の入力変数を、複数の状態として畳み込み、状態遷移を駆動する。

ai-desk の最深部の機構。

ai-desk 方法論 — 沖井広行・2026-05

# 1 分岐ではなく、列挙

if/else / switch を使わない。代わりに **全可能世界をデータとして列挙し、制約で filter**する。

## 従来 (分岐)

- 条件を **制御フロー** として書く
- 分岐ツリーで状態を辿る
- すべてのパスを脳内で追う必要
- 1箇所追加すると全体の整合性確認が必要
- 逆引き不可 (出力 → 入力は無理)

## CONSTRAINT FOLDING

- 条件を **データ構造** として持つ
- filter で生き残った世界が答え
- すべてのケースが 1箇所に並ぶ
- 1行追加すれば全体の整合性は構造的に保証
- 逆引き可能 (どの入力でその状態に行けるか)

## 2 パターン

「全可能性を data に展開」 → 「制約で filter」 → 「残った世界が結果」

```
// 1. 全可能世界を列挙する (data として)
const WORLDS = [
  { state: 'title',    input: 'start', next: 'playing' },
  { state: 'playing', input: 'pause', next: 'paused' },
  { state: 'playing', input: 'die',   next: 'gameover' },
  { state: 'paused',  input: 'resume', next: 'playing' },
  { state: 'gameover', input: 'reset', next: 'title' },
];

// 2. 制約で filter する
function reduce(state, input) {
  const world = WORLDS.find(w => w.state === state && w.input === input);
  return world ? world.next : state; // 該当世界がなければ現状維持
}
```

分岐ゼロ。データを変えるだけで状態機械が更新できる。

### 3 複数変数 → 複数状態

入力が増えるほど分岐は爆発する。畳み込みなら **データ次元が増えるだけ**。

```
// 例: 3 変数 (画面 / 入力 / 装備) で次状態を決める
const WORLDS = [
  { screen: 'playing', input: 'attack', weapon: 'sword', effect: 'slash', next: 'attacking' },
  { screen: 'playing', input: 'attack', weapon: 'bow', effect: 'shoot', next: 'shooting' },
  { screen: 'playing', input: 'attack', weapon: 'magic', effect: 'cast', next: 'casting' },
  { screen: 'paused', input: 'attack', weapon: '*', effect: null, next: 'paused' },
  // ...
];

function resolve(s, i, w) {
  return WORLDS.find(x =>
    x.screen === s && x.input === i && (x.weapon === w || x.weapon === '*')
  );
}
```

3 変数 × N 値の組合せが **テーブル 1 つ** に収まる。新しい武器を足す = 行を 1 個増やす、それだけ。

## 4 実例 1 — 画面遷移

`canvas-ui-sample.js` の画面遷移。if/else ゼロ。

```
const SCREEN_TRANSITIONS = [
  { from: 'title',    input: 'start', to: 'playing' },
  { from: 'playing',  input: 'pause', to: 'pause'   },
  { from: 'playing',  input: 'die',   to: 'gameover' },
  { from: 'pause',    input: 'resume', to: 'playing' },
  { from: 'pause',    input: 'quit',  to: 'title'   },
  { from: 'gameover', input: 'retry', to: 'playing' },
  { from: 'gameover', input: 'title', to: 'title'   },
];

function reduceScreen(state, input) {
  return SCREEN_TRANSITIONS.find(t => t.from === state && t.input === input)?.to ?? state;
}
```

遷移の **全ケースが 1 箇所**にある。AI が読むのもこの 7 行だけ。

遷移を追加する = 行を 1 つ増やす。制御フローを書き換える必要なし。

## 5 実例 2 — 27 世界の最小実証

`constraint-janken.js` — 3 人ジャンケン、全 27 世界。

```
const HANDS = ['rock', 'paper', 'scissors'];

// 全可能世界を列挙 (3 × 3 × 3 = 27 通り)
const WORLDS = HANDS.flatMap(a =>
  HANDS.flatMap(b =>
    HANDS.map(c => ({ a, b, c })))
);

// 制約で filter
function winners(state) {
  return WORLDS
    .filter(w => w.a === state.a && w.b === state.b && w.c === state.c)
    .map(w => judge(w)); // 勝者を出す
}
```

if 文ゼロで「全パターンを網羅した」状態判定が建つ。分岐の組合せ爆発が消える。

## 6 実例 3 — 1920 世界の実用規模

`fighter-cancel.test.js` — 格闘ゲームのキャンセル判定、全 1920 世界。

**1920**

全可能世界  
(状態 × 入力 × タイミング)

**19/19**

テスト合格  
全網羅

**1**

ファイル  
(全データが 1 箇所)

1920 世界全てが「正しい結果」と「実際の結果」が一致するか機械的に検査。

「全部試した」という保証 = 人間が一部読むより遥かに強い。

これが `fighter-cancel` の本質。

## 7 なぜこれが ai-desk の核か

Constraint Folding は「テクニック」ではない。計算の単位そのもの。

```
(state, command) => newState
```

この単一の式が、ai-desk のすべてに現れる。

- **L3 Logic 層 (Bible §2)**: 純粹 Reducer = まさにこの式
- **Twin (§4.5)**: 同じ式を別実装で再計算 → 突合せて検証
- **Event Sourcing (§5)**: state = events を全部この式で reduce した結果
- **状態遷移**: 上記の例の通り
- **網羅テスト**: 全可能 (state, command) で reduce して検証
- **逆引き**: 「このsを得るには、どの (state, command) があり得るか」

→ ai-desk が 5 公理から導出された方法論というより、1つのプリミティブから導出された体系。



## 8 これが可能にすること

### 網羅テスト

全可能世界を列挙してあるので、テストは「全行を回す」だけ。漏れが原理的に発生しない。

### 逆引き

出力 → 入力の検索が可能 (filter の方向を変えるだけ)。分岐ツリーでは原理的に不可能。

### 静的検証

「すべての state に対して入力が定義されているか」をデータの完全性で検査できる。

### 可視化

データを graph として描画可能。状態遷移図が自動生成される。

### AI による拡張

「新しい状態を追加して」 = AI が行を 1 つ追加する。コード書き換えゼロ、データ追記のみ。

### 仕様 = 実装

仕様書とコードが分離しない (データ自体が両方)。ドキュメント同期問題が消える。

## 9 なぜ AI と親和性が高いか

分岐 = 認知の分裂。畳み込み = 認知の集約。

- **分岐コードを読む AI**: 各 if/else を順に追い、メンタルモデルを構築。10 段ネストで context 食い尽くし
- **畳み込みコードを読む AI**: 1 つの table を見て全パターン一望。reduce 関数 1 つで挙動完全把握
- 分岐は **制御フロー = 時系列的**、AI は時系列追跡が苦手
- 畳み込みは **データ = 空間的**、AI は同時並行的処理が得意

Bible §0.0 「認知非対称性」の最も具体的な現れ。**分岐を畳み込みにする = 人間のためのコードを AI のためのコードにする。**

## 10 適用範囲

### 強く効く領域

- 有限離散状態 (UI 画面遷移 / ゲームロジック)
- 業務ルール (税計算 / 割引判定 / 送料)
- 権限・アクセス制御 (RBAC)
- 状態機械 (workflow / approval flow)
- 条件分岐の多いビジネスロジック

### 効かない領域

- 連続値の物理計算・補間
- リアルタイム描画ループ (60fps の中身)
- 木探索・グラフ探索 (将棋 AI の読み等)
- 機械学習の forward pass

アクションゲーム全体のバグ件数の **50-60%** がこの構造で構造的に絶滅する (PROMPT\_constraint\_folding 測定値)。

# ひとつの式

**(state, command) ⇒ newState**

これが ai-desk の **計算の単位**。

Reducer・Twin・Event Sourcing・状態遷移・網羅テスト・  
全部この式から導出される。

「畳み込み関数で複数変数を複数状態に展開して状態遷移する」

—— これが **分岐を捨てる** 覚悟そのもの。

[github.com/AoyamaRito/ai-desk](https://github.com/AoyamaRito/ai-desk)

沖井広行・蒼山りと